EL: An End-to-End AI-Augmented Pipeline with Human-in-the-Loop Validation for Trend-Driven Dropshipping Product Curation in the Indian E-Commerce Market

Divyesh Sharma
Department of Engineering
R.V. College of Engineering
Bengaluru, Karnataka, India
sharmadivyesh20070727@gmail.com

***Abstract*—The Indian dropshipping market is characterized by short demand cycles, vernacular trend signals, and a fragmented supplier landscape, making manual product research slow, inconsistent, and difficult to repeat at scale. We present *EL*, an end-to-end automation pipeline that ingests trending topics from three independent Indian sources (YouTube Trending, Google Trends RSS, and Google News RSS), scores each topic for purchase intent using a transparent three-tier keyword heuristic, dedupes near-identical topics through token-overlap fuzzy matching, and forwards the top-ranked subset to a large language model (LLM) agent for opportunity curation. The agent (Gemini 2.5 Flash) is augmented with a Tavily web-search tool and a Postgres-backed conversation memory to preserve cross-run context. Curated picks are routed to two parallel product-enrichment branches: a CJ Dropshipping API path and an Indian-marketplace path that combines Tavily search with a Browserbase scraping fallback and a Gemini-driven extraction step. Normalized product candidates are then assembled into Telegram review cards and dispatched to a human operator who approves, rejects, or skips each item; callbacks update Supabase through a dedicated trigger workflow. The pipeline is implemented on n8n Cloud as a 68-node workflow with two-layer error handling (per-node continueOnFail plus a workflow-level error subworkflow) and a 24-hour schedule. We report a case study on a production run from 2026-04-26 that produced 30 normalized product records spanning fashion, jewelry, and accessories categories, demonstrating end-to-end functionality. We discuss design trade-offs, three identified reliability issues, and the steps required to elevate the system to a benchmarked research contribution.**

***Index Terms*—workflow automation, dropshipping, large language model agents, human-in-the-loop, web scraping, e-commerce, intent classification, n8n, Gemini, India.**

I. Introduction

The Indian e-commerce ecosystem has undergone substantial structural change since 2020, driven by widespread smartphone penetration, the rapid expansion of digital payments, and the emergence of vernacular content as a primary discovery channel for consumer goods [1], [2]. Within this ecosystem, *dropshipping*—a retail fulfilment model in which a seller does not maintain inventory but instead forwards orders to a third-party supplier who ships directly to the end customer—has become a low-capital entry path for small and medium businesses (SMBs) and individual entrepreneurs [3], [4]. The defining operational challenge for any dropshipping seller is *product selection*: identifying, on a daily or near-daily cadence, items whose demand is currently rising, whose supplier price leaves room for margin, and whose logistics characteristics are compatible with the seller's target geography.

In Indian markets in particular, this selection process is complicated by three properties of the demand signal. First, trends are highly heterogeneous: a viral Bollywood track, a regional cricket fixture, and a state-specific festival can each spawn purchasing interest in entirely unrelated product categories within hours of each other. Second, signals arrive across multiple disconnected platforms (short-form video, news headlines, search queries) and rarely include a structured indication of purchase intent; a video title containing "wireless earbuds review" suggests intent very differently from a news headline containing the same phrase. Third, supplier catalogues—both global (CJ Dropshipping, AliExpress, Banggood) and Indian marketplaces (Amazon.in, Flipkart, Meesho)—use heterogeneous schemas, language conventions, and pricing formats that resist direct comparison without an explicit normalization layer.

Manual product research therefore typically involves an analyst rotating between a half-dozen browser tabs, copy-pasting candidate keywords into supplier search boxes, eyeballing prices and listings, and recording promising candidates into a spreadsheet. The process is slow, inconsistent across operators, and inherently non-reproducible. A daily refresh cycle for an SMB seller can easily consume two to four hours of analyst time before a single product is shortlisted for listing.

This paper presents *EL*[1] an automation system that compresses the daily research loop into an unattended pipeline. The system runs on n8n Cloud, executes once every 24 hours under a schedule trigger, and produces (i) a ranked list of trending Indian topics, (ii) an LLM-curated shortlist of dropshipping opportunities derived from those topics, (iii) a normalized product candidate set drawn from both supplier catalogues and Indian marketplace listings, and (iv) a human-in-the-loop (HIL) Telegram review surface through which an operator approves or rejects each candidate before it is committed as a final pick.

A. Contributions

The contributions of this paper are the following.

1) A *reproducible architecture* for trend-to-product dropshipping intelligence built entirely on managed cloud services (n8n, Supabase, Telegram), described at a level of detail that allows independent reconstruction. The complete workflow is exported as JSON and accompanies the paper as supplementary material.
2) A *hybrid intent-scoring algorithm* that combines a three-tier weighted keyword scheme with a 13-category product-mapping function and a fuzzy deduplication step based on token-overlap thresholding. The algorithm is transparent (no opaque model weights), inspectable, and deterministic for a fixed input.
3) A *multi-source product enrichment subsystem* that joins a structured supplier API (CJ Dropshipping) with an unstructured Indian-marketplace path. The marketplace path uses Tavily search as a primary channel and Browserbase as a fallback when search-result excerpts are too thin for downstream extraction, with a Gemini extraction step that converts raw HTML into a normalized product schema.
4) A *Human-in-the-Loop validation layer* delivered via Telegram inline-keyboard cards, with persistent state in Supabase, that closes the loop between automated curation and operator judgment. The HIL contract is formalized as a 26-field canonical schema (Table V).
5) A *two-layer error-handling discipline* combining per-node continue-on-fail behavior with a separate workflow-level error subworkflow, both of which surface failures to the operator on Telegram with execution-link traceability.
6) A *case-study evaluation* of a production run, in which the pipeline produced 30 normalized product candidates across fashion, jewelry, and accessory categories, providing descriptive evidence of end-to-end behavior.

[1]The project name *EL* is taken from the project's institutional context (the second-semester Experiential Learning course at R.V. College of Engineering); within the system documentation it is also expanded as "Emerging Lab." We retain the short form throughout this paper for brevity.

B. Scope and Non-Goals

This paper describes a working software prototype and a case study of one production run. It is *not* a controlled benchmark study: we do not claim that the hybrid intent scorer outperforms a pure-LLM baseline on a labeled dataset, and we do not provide statistical-significance guarantees on product-selection quality. Section XIII explicitly enumerates these limitations and outlines the steps required to elevate the work to a controlled comparative evaluation.

C. Paper Organization

The remainder of the paper is organized as follows. Section II surveys prior work on e-commerce automation, LLM agents, human-in-the-loop systems, intent classification, and resilient web scraping. Section III presents the high-level system architecture and design principles. Sections V to VIII describe the four pipeline phases in detail. Section IX treats error handling and observability. Section X covers implementation details, including the n8n node topology and the Supabase schema. Section XI reports the case-study results from the 2026-04-26 production run. Sections XII and XIII discuss findings and limitations. Section XIV concludes and outlines future work.

II. Background and Related Work

We organize the related literature into five strands that together motivate the design of EL.

A. E-Commerce Automation and Dropshipping

The dropshipping fulfilment model has been studied as a low-friction route into retail entrepreneurship since at least the late 2010s [3], [5]. Recent surveys document its rapid expansion in South and Southeast Asian markets, where SMB sellers leverage social-commerce channels (Instagram, WhatsApp, Telegram) for last-mile demand generation [4], [6]. Workflow-automation platforms such as Zapier, Make, and n8n have been positioned as enabling infrastructure for non-technical sellers to compose multi-API pipelines without writing back-end code [7], [8]. Compared to general no-code platforms, n8n distinguishes itself by exposing a fair-code license, supporting code-node JavaScript execution within the workflow graph, and providing first-class error-trigger semantics [8]—all of which EL relies on.

Prior work on supplier integration has largely focused on individual marketplaces (Amazon, AliExpress) rather than on cross-supplier orchestration [9], [10]. EL's combined CJ Dropshipping plus Indian-marketplace path is, to our knowledge, not directly addressed in the public literature, although the underlying technique—joining structured API output with unstructured scraped content—is well understood in adjacent domains such as price-comparison engines [11].

B. Trending-Topic Aggregation and Purchase-Intent Classification

Identifying topics with rising public attention is a long-studied problem in information retrieval and social-media mining [12], [13]. More recent work targets short-form video (YouTube, TikTok) as a leading-indicator channel for consumer demand [14], [15]. The downstream step of classifying a topic by *purchase intent*—rather than by sentiment or topicality—has been investigated using both supervised neural models [16], [17] and weakly-supervised keyword heuristics [18]. Hybrid approaches that combine keyword cues with model-based confidence have been shown to be both interpretable and competitive on small labeled corpora [19], motivating EL's three-tier keyword design.

C. LLM Agents for E-Commerce Tasks

The use of LLMs as agents—models that can plan, invoke external tools, and consume tool output as additional context—has emerged rapidly since 2023 [20]–[22]. Agent frameworks such as LangChain [23] and LlamaIndex [24] provide reference patterns for tool-use, retrieval-augmented generation, and persistent memory. Within e-commerce, LLM agents have been applied to product description generation [25], conversational shopping assistants [26], and—most relevant to this work—product opportunity scoring [27]. EL's agent design (Gemini 2.5 Flash plus Tavily search tool plus Postgres memory) follows the standard ReAct pattern [21], but composes it within an n8n workflow rather than within a Python agent runtime, which we discuss in Section VI.

D. Human-in-the-Loop Validation in AI Pipelines

Human-in-the-loop (HIL) machine learning is a well-established discipline [28], [29] that addresses the failure modes of fully-automated systems—hallucination, distributional shift, and silent error—by routing low-confidence or high-impact decisions to a human reviewer. In production e-commerce, HIL is commonly used for content moderation, price-anomaly review, and listing-quality control [30]. Telegram has emerged in recent years as an unusually convenient HIL surface for SMB operators because it is mobile-first, supports inline keyboards for one-tap callbacks, and exposes a permissive bot API [31]. EL adopts this pattern explicitly: each candidate product becomes a Telegram review card with Approve/Reject/Skip inline buttons, and the operator's tap is reflected back into Supabase via a dedicated callback workflow.

E. Resilient Web Scraping and Browserless Fallback

Web scraping at production reliability requires defending against rate limits, JavaScript-rendered content, anti-bot measures, and changing page structure [32], [33]. Recent industry tooling (Browserbase, Browserless, ScrapingBee) provides managed headless-browser sessions that abstract away these concerns at the cost of per-request fees [34]. A common pattern is to use a low-cost search-API channel (Tavily, Bing Web Search) as the primary path and to fall back to a managed browser only when the cheap path returns insufficient content [35]. EL implements this pattern with an explicit `If Tavily Content Thin` branch in the workflow graph, described in Section VII.

F. Workflow Orchestration Languages and Visual Programming

Workflow orchestration as a research area predates contemporary no-code platforms by several decades. Early work on scientific-workflow systems (Taverna, Kepler, Pegasus) established the directed-graph computation model and the separation between a workflow's static structure and its runtime execution [36], [37]. Visual programming for end-users has a parallel lineage in dataflow languages such as LabVIEW and Max/MSP [38]. Modern automation platforms (Zapier, Make, n8n, Workato) inherit both lineages: they expose a visual canvas in the dataflow tradition while running atop server-side orchestration engines descended from scientific-workflow systems [7], [8].

The trade-offs among these platforms have been characterized along several axes: ease of authoring (favoring fully-visual systems), expressive power (favoring code-augmented systems like n8n), portability (favoring self-hostable systems), and governance (favoring enterprise-managed systems) [39]. EL's choice of n8n reflects a preference for code-augmented expressiveness over fully-visual simplicity: the Phase 1 scoring algorithm (Section V-B) would be impractical to express through point-and-click composition.

G. Production Patterns for LLM-Augmented Systems

A growing literature documents the patterns that emerge when LLMs are placed in production pipelines rather than in research demos. Pattern catalogs have begun to consolidate around (i) prompt-as-code with versioning and CI [40], [41], (ii) tool-use through structured function-calling APIs [42], [43], (iii) retrieval-augmented generation with vector stores [44], [45], (iv) cost-aware fallback between expensive and cheap models [46], [47], and (v) human review as a fallback for low-confidence outputs [28], [30]. EL implements patterns (ii), (iv), and (v) in production form. We consider patterns (i) and (iii) as natural future extensions; the prompt is currently embedded as a fixed string in the agent node and is not version-controlled separately from the workflow JSON.

Together, these seven strands motivate a system that (i) orchestrates multiple managed services within a no-code workflow runtime, (ii) treats LLM agents as one component among many rather than as the primary interface, (iii) keeps a human in the validation loop, (iv) is resilient to partial failure, (v) inherits dataflow-orchestration semantics, and (vi) instantiates contemporary LLM-system production patterns. To our knowledge, no prior open work integrates all seven strands in a single end-to-end pipeline targeted at the Indian dropshipping market.

III. System Architecture

Figure 1 presents the high-level system architecture. EL consists of two cooperating workflows: the *main* EL workflow (68 nodes, of which 58 are functional and 10 are sticky-note documentation labels) and a separate *EL Error Handler* workflow (3 nodes) that is referenced from the main workflow's `errorWorkflow` setting and is invoked whenever the main workflow throws an unhandled error.

A. Design Principles

The architecture is shaped by four principles.

Phase tagging. The pipeline is organized into named phases (Phase 1 fetch, Phase 1 storage, Phase 2 AI curator, Phase 3a CJ supplier, Phase 3b Browserbase review, Phase 3c storage, Phase 4 candidate selection, Phase 4 Telegram delivery, Phase 6 Telegram callbacks, error handling) demarcated by sticky-note clusters on the n8n canvas. Each phase produces a clearly observable output (a sheet row, a JSON file, a database insert), which makes debugging tractable.

Storage redundancy. All meaningful intermediate state is written to at least two destinations: Google Sheets (operator-friendly, daily-tabbed), Google Drive (raw JSON archive), and—for product candidates—Supabase (queryable database). This redundancy means a failure in any single destination does not lose the run's data.

Per-node failure tolerance. Almost every functional node carries either `continueOnFail: true` or an explicit `onError: continueErrorOutput` configuration. Failures are routed to a central `Error Formatter` that emits a Telegram alert. This means a single misbehaving API does not cascade into a dead workflow.

Two-trigger separation. The main pipeline is driven by a 24-hour `Schedule` trigger, while the HIL callback path is driven by an independent `Telegram` trigger. The two paths share Supabase as their coordination substrate but otherwise execute on independent schedules.

B. Trigger Surfaces

The main workflow exposes two production triggers:

Schedule trigger. An n8n `scheduleTrigger` (v1.3) configured for a 24-hour interval, anchored to UTC. This trigger initiates the daily ingest-curate-enrich cycle.

Webhook trigger. A separate `webhook` trigger exposed at `/webhook/69d85d90-...-d4d8` responds immediately with the literal string "Workflow got started." and is used for manual on-demand initiation during development and demonstration runs.

The HIL callback path uses a third trigger, an n8n `telegramTrigger` bound to the "EL-HIL-BOT" credential, which fires whenever the operator interacts with the inline keyboard on a review card.

C. Service Inventory

Table I enumerates the external services consumed by the pipeline along with their role and the n8n credential under which they are configured.

Fig. 1. High-level architecture of the EL pipeline. Solid arrows denote primary data flow; dashed arrows denote error-routing paths to the formatter and the workflow-level error subworkflow. The Phase 3b marketplace path includes a conditional Browserbase fallback when the primary Tavily search returns thin content.

TABLE I
EXTERNAL SERVICES USED BY THE EL PIPELINE.

Service	Class	Role in pipeline	n8n credential
YouTube Data API v3	Trend source	Top 50 trending IN videos (Phase 1)	YouTube API Key
Google Trends RSS	Trend source	Daily trending searches, geo=IN (Phase 1)	— (open RSS)
Google News RSS	Trend source	Top 100 IN news headlines (Phase 1)	— (open RSS)
Gemini 2.5 Flash	LLM	Curation agent, product extraction	Gemini API Key (EL pipeline)
Tavily API	Search	Web search tool for agent and marketplace lookup	Tavily API (EL Phase 3)
Browserbase	Headless browser	Fallback fetch when Tavily content is thin	Browserbase API (EL Phase 3)
CJ Dropshipping API	Supplier catalog	Structured product search	— (inline auth)
Google Sheets	Operator UI	Daily ranked-trends and picks tabs	sharma divyesh api
Google Drive	Archive	Raw JSON dumps for trends and products	sharma divyesh
Supabase (Postgres)	DB	Product records, agent memory, HIL state	Supabase yatralounge (Dropship Memory)
Telegram (Developer)	Alerts	Per-node failure notifications	EL-DEVELOPER-ALERT
Telegram (HIL)	Operator UI	Review cards with inline approval keyboard	EL-HIL-BOT

IV. Engineering Patterns and Cross-Cutting Decisions

Before describing the four pipeline phases in detail (Sections V to VIII), we surface several cross-cutting engineering patterns that are reused throughout the workflow. Treating these as named patterns rather than as ad-hoc node configurations was instrumental in keeping the 68-node workflow comprehensible.

A. Credentials and Secrets Management

n8n provides a credentials store that abstracts authentication material away from individual node configurations. EL uses one credential per external service, named according to the convention `<service> <purpose>` (e.g., "Supabase yatralounge (Dropship Memory)"). All HTTP nodes that call a managed API reference these credentials by name rather than embedding tokens in the node body. The CJ Dropshipping email-plus-API-key pair is a partial exception: it is stored as inline text in the `CJ Get Token` node because the CJ API does not conform to any of n8n's pre-built authentication patterns. This is acceptable for a prototype but would be moved to a custom credential type for production.

The `.env` file in the repository root contains placeholder names rather than secrets, and the actual values live only in the n8n cloud account and the Supabase dashboard. The repository's `.gitignore` prevents accidental commits of the populated `.env`.

B. Idempotency at the Persistence Boundary

The persistence boundary—between the volatile in-flight workflow state and the durable storage destinations—is the most failure-prone segment of any automation pipeline. A retry of a partially-completed run must not produce duplicate rows, double-spent API calls, or two-fold Telegram messages.

EL addresses this with three idempotency mechanisms. First, each Supabase insert is implemented as an upsert on a deliberately-chosen unique key: `(product_url, source_topic, run_date)` for `scraped_products`, and `(workflow_run_id, source_provider, product_url)` for `hil_reviews`. Second, each Google Sheets write to a date-tabbed sheet uses `appendOrUpdate` semantics: a re-run of the same logical day appends to the existing tab rather than overwriting it. Third, each Telegram delivery is gated by a Supabase write that records the Telegram `message_id`; a candidate already marked as "delivered" is not re-delivered on retry.

C. Observability Beyond Errors

The error-handling discipline (Section IX) covers failure modes, but reliable operation also requires positive observability of the happy path. EL relies on three signals.

- n8n execution log. Every successful run produces a structured execution record visible in the n8n UI, with per-node timing, payload sizes, and (optionally) full payload snapshots. This is the primary post-hoc debugging surface.
- Date-tabbed sheets. The presence (or absence) of a freshly-created date tab in the trends spreadsheet is a coarse-grained but immediately visible health signal: an operator opening the sheet at 09:00 IST can verify the daily run completed by checking for the day's tab.
- Telegram delivery cadence. The arrival of HIL review cards on the operator's Telegram chat at the expected time of day is itself a liveness signal. A missing batch indicates that the upstream pipeline failed or that the Telegram-delivery sub-flow is broken.

We deliberately do *not* introduce a dedicated metrics or tracing layer (Prometheus, Grafana, OpenTelemetry). For a prototype operating at the daily granularity of a single

TABLE II
COMPARISON OF WORKFLOW-AUTOMATION PLATFORMS CONSIDERED FOR EL.

Property	n8n	Zapier	Make	Py
Self-hostable	✓	—	—	✓
Code-node JS	✓	lim.	lim.	N/A
LangChain integration	✓	—	—	✓
Error-trigger workflow	✓	lim.	lim.	man.
Free tier daily run	✓	—	lim.	✓
Visual canvas	✓	✓	✓	—
Time to first run	low	low	low	high

SMB seller, the operational complexity of these tools is unjustified.

D. Comparison with Alternative Workflow Platforms

We considered three alternatives to n8n during the planning phase. Table II summarizes the comparison.

n8n was selected primarily because its code-node JavaScript and first-class error-trigger semantics together cover the spectrum from "drag-and-drop simple integration" to "arbitrary computation when needed," without forcing a switch between two platforms as a project grows.

V. Phase 1: Multi-Source Trend Aggregation and Hybrid Intent Scoring

Phase 1 ingests trending topics from three independent Indian sources, scores each topic for purchase intent, deduplicates near-identical topics, and produces a ranked candidate list for the curation agent.

A. Source Selection and Justification

We use three sources rather than one for two reasons. First, each source has a distinct demand-signal latency: YouTube Trending tends to surface entertainment-driven attention with a lag of hours; Google Trends RSS captures search-driven curiosity within minutes; Google News RSS captures editorial-mediated topics on a publishing cycle of tens of minutes to hours. Combining the three reduces the variance of the daily candidate set. Second, source diversity provides resilience: when a single source returns an HTTP error or schema change (which empirically happens several times per quarter), the other two still produce candidates and the pipeline degrades gracefully rather than failing.

- YouTube Trending: Fetched via the YouTube Data API v3 `videos` endpoint with parameters `chart=mostPopular`, `regionCode=IN`, `maxResults=50`, `part=snippet`. Authentication is via API key. Each item contributes its video title as the topic and the (often empty) tag list as the related-queries hint.
- Google Trends RSS: Fetched via the public RSS feed at `trends.google.com/.../daily?geo=IN`. Item titles are extracted via regex from `<title>` elements in the XML (the first item is the channel title and is skipped).
- Google News RSS: Fetched via the public Indian-edition RSS feed (`hl=en-IN, gl=IN, ceid=IN:en`) with the first 100 items consumed. Title extraction uses the same regex strategy.

All three fetches are performed inside a single n8n Code node (`Fetch . Score . Dedupe . Rank`) using the `$http` client; each fetch is wrapped in a `try/catch` so that a failure in one source produces a logged warning rather than aborting the others.

B. Three-Tier Intent Scoring

Each topic is scored for purchase intent using a transparent additive scheme over three keyword tiers and one negative tier. The tiers are calibrated empirically:

- T1 (commercial, weight 0.30): strong direct purchase signals such as "buy", "price", "cheap", "best", "review", "order", "shop", "deal", "discount", "sale", "offer", "coupon", "free shipping", "cash on delivery", "lowest price", "wholesale", "under 500", "under 1000".
- T2 (research, weight 0.15): indirect signals such as research cues such as "alternative to", "similar to", "specs", "features", "unboxing", "pros and cons", "is it worth it", "buying guide", "top 10", "tier list", "benchmarks", "vs", "comparison".
- T3 (product context, weight 0.10): product-context terms such as "wireless", "portable", "new", "latest", "trending", "india 2026", "original", "authentic", "warranty", "return policy", "flash sale", "pre-order", "launch date", "near me".
- NEG (negative cues, weight −0.20): phrases that suggest non-purchase intent such as "how to", "tutorial", "diy", "repair", "fix", "not working", "error", "broken", "manual pdf", "driver download", "free download".

The composite score is computed as in Equation (1) and then clamped to $[0, 1]$ and rounded to three decimals:

$$s(t) = \mathrm{clamp}_{[0,1]}\Big(\sum_{k \in T1} 0.30 \mathbb{1}[k \in h(t)] + \sum_{k \in T2} 0.15 \mathbb{1}[k \in h(t)] + \sum_{k \in T3} 0.10 \mathbb{1}[k \in h(t)] - \sum_{k \in N} 0.20 \mathbb{1}[k \in h(t)] \Big), \tag{1}$$

where t is a topic, $h(t)$ is the lowercased concatenation of the topic and its related-queries hint, and $\mathbb{1}[\cdot]$ is the indicator function for substring membership.

The choice of an additive, weighted, threshold-saturated scheme rather than a learned model is deliberate. It is interpretable (an operator can audit why a score was assigned), deterministic (the same input always produces the same score), and cost-free at runtime (no model inference). Its principal limitation is that it does not generalize beyond its keyword vocabulary; this is partially mitigated by the downstream LLM agent (Section VI) which is not constrained to the keyword view.

TABLE III
PRODUCT CATEGORIES USED IN PHASE 1 MAPPING. REPRESENTATIVE KEYWORDS SHOWN; FULL DICTIONARY CONTAINS 14–17 KEYWORDS PER CATEGORY.

Category	Representative keywords
Electronics	wireless, bluetooth, earbuds, phone, laptop, charger, smartwatch
Fashion	shirt, dress, kurta, saree, sneakers, watch, bag, sunglasses
Home	kitchen, decor, organizer, fan, cooler, bedsheet, cookware
Fitness	gym, yoga, protein, supplement, dumbbells, treadmill
Beauty	skincare, hair, serum, moisturizer, sunscreen, perfume
Accessories	case, cover, stand, mount, strap, tempered glass
Automotive	car, bike, helmet, dashcam, tyre, wax, polish
Baby & kids	toy, puzzle, lego, diaper, stroller, pacifier
Pets	dog, cat, food, collar, leash, aquarium, grooming
Office & stationery	book, notebook, pen, desk, chair, printer
Health & medical	thermometer, vitamins, mask, sanitizer, oximeter
Tools & hardware	drill, screwdriver, wrench, saw, hammer, multimeter
Grocery & food	coffee, tea, snacks, chocolate, dry fruits, oil, spices

Algorithm 1 Token-overlap fuzzy deduplication

Require: Candidate list C, stop-word set W, threshold $\tau = 0.70$

$K \leftarrow$ empty list of kept candidates
for all $c \in C$ **do**
  $w_c \leftarrow$ tokens of c.topic minus W
  if $w_c = \emptyset$ **then**
    continue
  end if
  $d \leftarrow -1$ {index of duplicate match}
  for all $k \in K$ **do**
    $w_k \leftarrow$ tokens of k.topic minus W
    $o \leftarrow |w_c \cap w_k|$
    $m \leftarrow \min(|w_c|, |w_k|)$
    if $m > 0$ **and** $o/m > \tau$ **then**
      $d \leftarrow$ index(k) in K; **break**
    end if
  end for
  if $d = -1$ **then**
    append c to K
  else if $|c$.related$| > |K[d]$.related$|$ **then**
    $K[d] \leftarrow c$
  end if
end for
return K

C. Category Mapping

In parallel with intent scoring, each topic is mapped to one or more of 13 product categories using a per-category keyword dictionary. Table III summarizes the categories and a representative subset of their keywords; the full dictionary contains 14 to 17 keywords per category.

The mapping is a simple word-boundary regex match. A topic that matches no category is labelled "uncategorized" and is retained (downstream the LLM agent may still find an opportunity in it). The category labels feed directly into the Phase 2 prompt as additional context, enabling the agent to apply category-specific reasoning.

D. Fuzzy Deduplication

Three independent sources frequently surface the same underlying topic in slightly different wording ("Wireless earbuds review" vs. "Best wireless earbuds 2026 review"). To prevent the agent's input from being dominated by such near-duplicates, we apply a fuzzy deduplication step described in Algorithm 1.

The threshold $\tau = 0.70$ is empirically chosen: lower values collapse semantically distinct topics that happen to share filler words; higher values fail to merge near-identical reformulations. The retained representative is the candidate with the longer related-queries hint, on the heuristic that more context improves downstream prompting.

After deduplication the candidates are sorted by descending intent score and assigned a final integer rank. The complete Phase 1 output is a JSON document with a `metadata` block (timestamp, geo, total count, sources used) and a `trends` array of ranked items, each with topic, score, source, related queries, and suggested categories.

E. Storage Fan-Out

The ranked output fans out to two destinations. A `Create Day Tab` node creates (or no-ops on) a Google Sheets tab named with the current date in `YYYY-MM-DD` format inside spreadsheet `1WVI...Ln2A`. A `Prepare Sheet Rows` node converts the JSON array to row tuples and `Write Rows to Sheet` appends them. In parallel, a `Prepare JSON File` node writes a Drive upload payload and `Drive Upload` stores the raw JSON in folder `1MOF...tjVja` under the filename `trending_india_YYYY-MM-DD.json`.

VI. PHASE 2: LLM AGENT FOR TOPIC CURATION

Phase 2 receives the top 30 ranked topics from Phase 1 and converts them into a smaller, structured list of dropshipping opportunities suitable for downstream supplier search. The phase is implemented as a single n8n LangChain Agent node (`Dropship AI Agent`) whose three sub-components—a chat model, a memory store, and a tool—are wired into the agent through n8n's LangChain integration.

A. Agent Composition

Chat model: Gemini 2.5 Flash, accessed through Google's Gemini API. Flash is selected over Pro for its

lower latency and order-of-magnitude lower per-token cost; the curation task does not require Pro-level reasoning.

Memory A Postgres-backed chat memory bound to the project's Supabase instance. The agent can therefore reference its prior runs (yesterday's picks, recurrent topics, prior reasoning) when curating today's list, which empirically reduces oscillation between similar candidates day to day.

Tool A Tavily web-search tool exposed to the agent through n8n's `toolCode` node. The agent invokes the tool to verify demand for a candidate (e.g., to confirm that an earbuds-related topic still has active product interest) before committing it to the output list.

B. Prompt Design

The agent receives a system prompt that establishes its role ("a dropshipping intelligence analyst for the Indian market") and a strict output contract requiring valid JSON. The user message contains the top-30 topic list with their scores, sources, and suggested categories. The agent is instructed to (i) consider the demand strength implied by source diversity and intent score, (ii) optionally call the Tavily tool to verify ongoing demand, (iii) consider whether the topic maps to a product that can plausibly be dropshipped from CJ catalogues or Indian marketplaces, and (iv) return at most 10 picks ranked by an opportunity score on a 0–10 scale.

The strict JSON requirement is enforced both in the prompt and downstream by the `Parse Agent Output` node, which uses a JSON-extraction regex with a Markdown-fence-stripping fallback. If parsing fails, the node routes to the error path and the operator is alerted on Telegram.

C. Output Contract and Fan-Out

Each pick produced by the agent has the following fields: `rank`, `topic` (lifted from Phase 1), `opportunity_score` (0–10), `reason` (a short free-text rationale), `product_type` (a category label, often but not always one of Phase 1's 13 categories), and `target_audience`. The picks are then fanned out to five downstream nodes simultaneously: `Write Curated Picks` (Google Sheets), `CJ Get Token` (initiates the supplier path), `Build Search Query` (prepares the supplier search keywords), `Build Tavily Query` (initiates the marketplace path), and `Error Formatter` (passive, on-error only).

Two design observations are worth noting. First, the simultaneous fan-out triggers both product-enrichment paths (CJ and marketplace) for every pick, deliberately maximizing the candidate set at the expense of API cost. A future cost-aware variant would gate one path behind the other's failure. Second, the `CJ Get Token` node currently fans only to `Error Formatter`, not to `CJ Product List`; the token is therefore obtained but is not wired into the subsequent product-list call. We treat this as a known issue (Section XIII, item 1) rather than as intended behavior.

VII. Phase 3: Multi-Source Product Enrichment

Phase 3 converts each curated pick into one or more concrete product candidates. It comprises three sub-paths: a CJ Dropshipping supplier path (3a), an Indian-marketplace path with Browserbase fallback and Gemini extraction (3b), and a unified storage fan-out (3c).

A. Phase 3a: CJ Dropshipping Supplier Path

The CJ path is the structured, lower-variance path. It uses the CJ Dropshipping product-list API directly.

CJ Get Token Performs an HTTP POST to the CJ access-token endpoint (`developers.cjdropshipping.com-/api/getAccessToken`) with the configured email and API-key pair. Returns a short-lived `accessToken` (TTL on the order of hours) which is required as the `CJ-Access-Token` header on subsequent product calls.

Build Search Query A Code node that derives concise product-noun search keywords from the agent's pick topic. The transformation strips brand-name modifiers, video-title decorations, and Indian-language fragments, leaving a 1–3 word product noun phrase suitable for the CJ search index. Pagination parameters (`pageNum=1`, `pageSize=50`) are also set here.

CJ Product List An HTTP GET to the CJ products endpoint with the access-token header. Returns a paginated list of product records with fields including `pid`, `productNameEn`, `sellPrice`, `productImage`, `listedNum` (an estimate of how many sellers have listed the product, used as a demand proxy), `categoryName`, and `shippingCountryCodes`.

Pick Top 3 A Code node that ranks the returned products by `listedNum` in descending order and selects the top three for downstream processing. The intuition is that products with many listings are more likely to be currently selling, which is a stronger prior than CJ's default relevance ranking.

B. Phase 3b: Indian-Marketplace Path with Browserbase Fallback

The marketplace path is the unstructured, higher-variance path. Its goal is to surface products that are already available on Indian-facing marketplaces (Amazon.in, Flipkart, Meesho), which CJ's catalogue does not cover.

Build Tavily Query Constructs a search query of the form "⟨pick topic⟩ buy site:amazon.in OR site:flipkart.com", with simple cleanup applied to the topic.

Tavily Search (IN Market) Performs an HTTP POST to the Tavily search API with `search_depth=advanced` and an explicit `include_domains` filter restricted to Indian marketplaces. Returns a list of result objects each containing a URL, title, snippet, and (optionally) raw page content.

Pick Indian Candidates A Code node that filters the Tavily results to those whose URL host belongs to a recognized Indian marketplace and that retains the most relevant fields for downstream extraction.

If Tavily Content Thin An IF node whose predicate tests whether the Tavily result's raw-content field exceeds a minimum length threshold. The branch decides whether to invoke Browserbase or to bypass it.

Browserbase Fetch When Tavily content is thin, this node performs an HTTP GET to the Browserbase fetch API which returns the rendered HTML of the listing page. Browserbase handles JavaScript rendering, cookie state, and basic anti-bot evasion.

Strip HTML A Code node that converts the raw HTML to a compact text representation by removing script/style blocks, collapsing whitespace, and truncating to a token budget compatible with the downstream Gemini call.

Prepare Gemini Prompt Constructs a structured extraction prompt around the cleaned content, asking Gemini to return a JSON object with product name, price, rating, review count, image URL, and availability status.

Gemini Extract Product An HTTP POST to the Gemini API. The response is parsed and routed to `Normalize Browserbase Review`, which conforms the extracted record to the canonical HIL contract (Section VIII).

The fallback design is motivated by cost. Tavily provides a search-and-snippet response at a cost per query that is approximately one order of magnitude lower than a Browserbase session. By gating Browserbase behind an explicit thin-content test, we use the expensive path only when the cheap path is insufficient.

C. Phase 3c: Normalization and Storage Fan-Out

The normalized records from both paths are merged at `Merge Sources`, which produces a single stream of product candidates. The stream then fans out to four destinations:

- `Create Day Tab (Scraped)`: ensures a date-named tab exists in spreadsheet `1lLm...IiieQ`.
- `Prepare Sheet Rows (Scraped)` → `Sheet Append (Scraped)`: writes the candidate rows for operator visibility.
- `Bundle JSON (Scraped)` → `Drive Upload (Scraped)`: archives the raw JSON to Drive folder `1jih...Pfvhx`.
- `Supabase Insert (Scraped)`: upserts the records into the `scraped_products` Postgres table, with the upsert key `(product_url, source_topic)` ensuring that a product surfaced by two distinct topics is recorded once but with both topic associations.

In parallel, the records emerging from the normalize-and-merge step on the review path also flow into `Supabase Insert (HIL Reviews)`, which writes to a separate `hil_reviews` table. This separation preserves the storage-row schema (which is tuned for analytical querying) independently of the HIL contract (which is tuned for operator review).

VIII. Phase 4: Human-in-the-Loop Telegram Validation

Phase 4 wraps each product candidate in an operator-friendly Telegram message and accepts the operator's approval, rejection, or skip via inline-keyboard callbacks.

A. Candidate Selection

Before any candidate is delivered to the operator, a candidate-selection step trims the pool to a manageable daily volume. Table IV lists the thresholds used.

The selector is a small JavaScript module (`scripts/build_phase4_candidate_selection.js`) that applies the thresholds in two passes: a strict pass at `MIN_SCORE`, and—if that pass yields too few candidates—a fallback pass at `FALLBACK_SCORE`. Per-topic and per-source caps are enforced by greedy descent on opportunity score.

B. Canonical HIL Contract

Table V formalizes the 26-field HIL contract that every candidate must satisfy before entering the review queue. The contract is defined in `docs/hil-review-contract.md` as version `hil_v1`.

The contract enforces four normalization invariants. First, the rating scale is normalized to 0–5 numeric, with unknown ratings represented by `null` rather than a placeholder string. Second, both `price_text` and `price_numeric` are preserved (the textual form is what the operator needs to read; the numeric form is what downstream filtering needs to reason about). Third, `image_url` must be the first usable element of `image_urls`. Fourth, missing optional fields are `null`, not the string "N/A".

C. Telegram Delivery

The delivery sub-flow proceeds as follows. `Prepare Telegram Card` composes the message: a Markdown caption with title, price, source topic, opportunity score, and a clickable product URL; an inline keyboard with four buttons (`Approve`, `Reject`, `Skip`, `Open Product`); and the primary image URL. `Download Product Image` fetches the image as binary, with a User-Agent header set to a desktop browser string to avoid common 403 responses.

`Send HIL Telegram Photo` delivers the message via the `sendPhoto` Telegram Bot API method. On success, `Mark Telegram Photo Sent` writes the Telegram message ID and timestamp into Supabase (`hil_reviews.telegram_message_id`). On failure—commonly because the image URL is hot-link protected or returns non-image content—the flow falls through to `Send HIL Telegram Text Fallback`, which delivers a text-only card via `sendMessage`, and `Mark Telegram Text Fallback` records the fallback in Supabase.

We note here a known issue (Section XIII, item 2): `Download Product Image` currently wires to *both* the

TABLE IV
PHASE 4 CANDIDATE-SELECTION THRESHOLDS.

Threshold	Value	Purpose
`MIN_SCORE`	25	Minimum opportunity score (out of 100) for a candidate to be eligible.
`FALLBACK_SCORE`	20	Used if the MIN_SCORE filter yields fewer than `MIN_DAILY` picks.
`TOPIC_CAP`	2	Maximum candidates per source topic, to prevent topic monopolization.
`CJ_CAP`	5	Maximum CJ-sourced candidates per day.
`BROWSERBASE_CAP`	7	Maximum marketplace-sourced candidates per day.
`TOTAL_CAP`	10	Hard upper bound on daily HIL volume.

photo path and the text-fallback path, rather than gating the fallback behind the photo path's failure. Empirically this produces a duplicate text card alongside each successful photo card. The fix is to remove the unconditional fallback edge and re-route it through the error output of `Send HIL Telegram Photo`.

D. Callback Handling (Phase 6)

Operator interactions with the inline keyboard are received by an independent sub-workflow rooted at the `Telegram Trigger` (Phase 6). The trigger fires on `callback_query` events from the EL-HIL-BOT bot.

- Parse HIL Callback — Extracts the callback payload, including the operator's choice (`approve`, `reject`, `skip`) and the Supabase row identifier embedded in the callback data.
- Apply HIL Callback — Performs a UPDATE on the `hil_reviews` row, setting `approval_status`, `approval_decided_at`, and `approval_actor` (the Telegram user). The update is idempotent: a second tap has no further effect.
- Answer HIL Callback — Calls Telegram's `answerCallbackQuery` method to dismiss the loading spinner on the operator's tap.
- If Callback Finalized — Branches on whether the final state is non-pending. If so:
  - Edit HIL Review Message — Edits the original preview card to display the final state (e.g., "✓ Approved by @divyesh"). On success, `Log HIL Message Edited` records the edit in Supabase.
  - Delete HIL Review Message — If the edit fails (e.g., because the message is too old to edit per Telegram's 48-hour limit), the original card is deleted to prevent stale state in the operator's chat. `Log HIL Message Deleted` records the deletion.

The callback path is structurally interesting because it is one of the few production trigger surfaces that is *not* a schedule or webhook; it is event-driven by operator action. n8n's Telegram trigger handles long-polling internally so no external infrastructure is required.

E. Bayesian HIL Feedback Calibration

The mechanisms described in Sections VIII-A, VIII-B and VIII-D reduce the human reviewer to a one-way gate: a tap either admits a candidate into the catalogue or removes it. The information content of that tap—whether the operator agreed with the agent's curation for this particular category and price range—is then discarded. Recycling such operator decisions back into a learning loop has been shown to substantially improve agent quality across very different domains, from preference learning for large language models [48] to mixed-initiative planning systems [49]; we propose a smaller and more conservative variant suitable for the data volume available to a single-operator HIL pipeline. The construction is henceforth referred to as *BCC-HIL* (Bayesian Calibration with Conjugate HIL feedback). Because the Beta–Bernoulli pair is conjugate [50], both the posterior update and the inference-time score are closed-form scalar operations, and the addition therefore introduces no measurable runtime overhead.

1) Formal model: Let $\mathcal{C}$ denote the set of 13 product categories enumerated by the Phase-1 keyword classifier (Section V-C), and let $y \in \{0, 1\}$ denote the binary outcome of a single Telegram callback (1 for *approve*, 0 for *reject*; the *skip* decision is treated as missing data and produces no update). For each category $c \in \mathcal{C}$, BCC-HIL maintains a Beta posterior

$$\theta_c \mid \mathcal{D}_c \;\sim\; \mathrm{Beta}(\alpha_c,\, \beta_c), \tag{2}$$

where $\mathcal{D}_c = \{y_1, \ldots, y_{n_c}\}$ are the HIL outcomes recorded for products in category c, and the hyperparameters (α_c, β_c) are initialized at $(1, 1)$, encoding a uniform prior over the per-category approval rate. Each new observation y in category c triggers the closed-form update

$$(\alpha_c, \beta_c) \;\leftarrow\; (\alpha_c + y,\; \beta_c + (1 - y)), \tag{3}$$

which preserves conjugacy.

The posterior mean $\hat{\mu}_c = \alpha_c/(\alpha_c + \beta_c)$ is the Bayes-optimal point estimate of the per-category approval rate under squared-error loss. We combine it with the LLM agent's raw opportunity score $s \in [0, 10]$ via empirical-Bayes shrinkage [51]:

$$\mathrm{cal}(p) \;=\; w_c \cdot \hat{\mu}_c \;+\; (1 - w_c) \cdot \frac{s}{10}, \qquad w_c \;=\; \frac{n_c}{n_c + n_0}, \tag{4}$$

where $n_c = (\alpha_c + \beta_c) - 2$ is the effective sample size beyond the prior and n_0 is a smoothing hyperparameter that controls how aggressively the calibrator overrides the agent.

2) Cold-start and convergence properties: At cold start, $n_c = 0$ implies $w_c = 0$ and the calibrated score collapses to $s/10$, the simple normalization of the raw score; the calibrator is a no-op until feedback exists. As feedback accumulates, w_c grows monotonically and asymptotes to

TABLE V
CANONICAL HUMAN-IN-THE-LOOP REVIEW CONTRACT (`hil_v1`).

Field	Type	Req.	Description
review_schema_version	string	✓	Contract version, fixed to `hil_v1`.
workflow_name	string	✓	Workflow identifier, fixed to `EL`.
workflow_run_id	string	✓	Execution identifier or deterministic run key.
run_date	string	✓	Logical run date (`YYYY-MM-DD`).
source_provider	string	✓	`cj_dropshipping` or `browserbase_marketplace`.
source_topic	string	✓	Curated trend topic that produced the candidate.
source_pick_rank	number/null		Rank in the upstream curated pick list.
opportunity_score	number/null		Upstream AI opportunity score (0–10).
product_name	string	✓	Human-readable product title for Telegram caption.
product_url	string	✓	Product page URL for the operator.
product_sku	string/null		Supplier SKU.
price_text	string/null		Source-native price string.
price_numeric	number/null		Parsed numeric price for ranking.
currency	string/null		ISO-like currency label (`INR`, `USD`).
product_rating	number/null		Top-level rating normalized to 0–5.
reviews_count	number/null		Review or rating count.
image_url	string/null		Primary image URL for Telegram delivery.
image_urls	array	✓	Ordered image URL list (may be empty).
description	string/null		Operator-facing summary, ≤ 300 chars.
supplier_name	string/null		Supplier or seller name.
marketplace	string/null		Marketplace host (e.g., `amazon.in`).
availability_status	string/null		One of `in_stock`, `out_of_stock`, `unknown`.
approval_status	string	✓	Initial state, fixed to `pending`.
approval_channel	string	✓	Initial channel, fixed to `telegram`.
raw_payload	object/string	✓	Original source payload for audit.
scraped_at	string	✓	ISO timestamp of candidate production.

one, so the calibrated score is increasingly dominated by the empirical approval rate. The crossover at $w_c = 0.5$ occurs at $n_c = n_0$, which determines the calibrator's reactivity. We default to $n_0 = 5$, chosen so that approximately one full HIL batch (the daily output of the pipeline is around ten cards, of which roughly five to seven are non-skipped per the anecdotal observations of Section XI-F) is required before the posterior carries majority weight against the agent. Smaller n_0 produces a more reactive calibrator at the cost of higher variance under sparse feedback; larger n_0 effectively distrusts the HIL signal in favor of the agent.

3) Implementation: The calibrator is implemented as a self-contained Python module (`scripts/bayesian_calibration.py`) of approximately 110 lines with no third-party dependencies, accompanied by a unit-test suite of 14 cases that covers four properties: cold-start equivalence to the normalized raw score, convergence of $\hat{\mu}_c$ to the empirical approval rate after long runs of identical outcomes, monotonicity of w_c in n_c, and exact JSON round-trip persistence of the per-category state. The same construction has also been wired into the live n8n workflow as one Postgres read node (`Read BCC Posteriors`), one JavaScript Code node (`BCC Calibrate Selection`) inserted between Phase-4 candidate selection and the HIL persistence step, and one Postgres update node (`Update BCC Posterior`) attached to the Phase-6 callback handler; the per-category state is persisted in a new `private.bcc_posteriors` table. The integration is described concretely in Section X-F and Section F. The empirical behavior of the calibrator on the case-study dataset is reported in Section XI-G.

IX. ERROR HANDLING AND OBSERVABILITY

EL implements a two-layer error-handling discipline, each layer addressing a different failure mode.

A. Layer 1: Per-Node Continue-on-Fail

Almost every functional node carries either `continueOnFail: true` (storage nodes, Telegram nodes, retry-tolerant HTTP nodes) or an explicit `onError: continueErrorOutput` (most Code nodes and primary HTTP nodes). The design intent differs between the two:

- `continueOnFail` is used where a single-item failure should not abort the batch, but the failure does not need to be reported (e.g., a single Sheets write that fails because of a transient quota error is silently retried on the next run).
- `continueErrorOutput` means a failure must be visible: the error item is routed out the node's error port to a central `Error Formatter` Code node, which composes a Markdown-formatted Telegram message containing the failed node name, the error message, an Indian Standard Time (IST) timestamp, and an n8n execution-link URL.

The `Error Formatter` pipes its output to `Telegram Alert`, which sends to the `EL-DEVELOPER-ALERT` Telegram credential. The operator therefore receives a per-node failure notification within seconds of the error.

B. Layer 2: Workflow-Level Error Subworkflow

A separate workflow (`EL Error Handler`, ID `rE91...8P`) is registered as the main workflow's

`errorWorkflow` setting. Whenever the main workflow throws an unhandled exception—a class of failure not caught by Layer 1, such as a credential-expiry error during workflow start—n8n invokes the error workflow with an `errorTrigger` payload containing the failing node, the error message, the execution ID, and the workflow ID.

The error subworkflow is intentionally minimal. Listing 1 shows the full Code-node implementation, which produces a Telegram-Markdown message identical in shape to the Layer-1 message and dispatches it to the same developer chat.

Listing 1. Error Formatter (Layer 2). Reduced from the production code to remove credentials and shorten string concatenations.

```
const d = $input.first().json;
const err     = d?.execution?.error || {};
const nodeName = err?.node?.name
              || err?.nodeName
              || 'Unknown Node';
const msg = (err?.message
          || err?.description
          || JSON.stringify(err)
            ).substring(0, 400);
const ts = new Date().toLocaleString(
  'en-IN', { timeZone: 'Asia/Kolkata' });
const wfId = d?.workflow?.id
          || '298f3JSOE72nhW59';
const execId = d?.execution?.id || '';
const url = execId
  ? `https://divyesh-sharma.app.n8n.cloud/`
   +`workflow/${wfId}/executions/${execId}`
  : `https://divyesh-sharma.app.n8n.cloud/`
   +`workflow/${wfId}/executions`;
const text =
  `EL Node Failed\n\n`+
  `Node: ${nodeName}\n`+
  `Error: ${msg}\n`+
  `Time (IST): ${ts}\n`+
  `[View Execution](${url})`;
return [{ json: { text } }];
```

The two layers are intentionally redundant: Layer 1 catches errors that the main workflow continues past; Layer 2 catches errors that the main workflow cannot continue past. Together they ensure that no failure mode is silent.

X. Implementation and Deployment

A. Runtime

The pipeline runs on n8n Cloud. n8n is a fair-code workflow-automation runtime that executes a directed graph of typed nodes, where each node is either a built-in connector (HTTP Request, Code, IF, Merge, Schedule, etc.) or a domain-specific connector (Google Sheets, Drive, Postgres, Telegram, Gemini, etc.) [8]. A node executes when all its inputs are satisfied; the runtime supports both synchronous fan-out and asynchronous trigger nodes (Schedule, Webhook, Telegram) on independent execution contexts.

B. Workflow Structure

The main workflow contains 68 total nodes, of which 10 are sticky-note documentation labels (used to mark phase boundaries on the canvas) and 58 are functional. Table VI breaks down the functional nodes by type.

The connection graph contains 39 inter-node edges, including the error-routing edges to `Error Formatter`.

TABLE VI
Functional node counts in the main EL workflow by type.

Node type	Count
Code (JavaScript)	21
HTTP Request	9
Google Sheets	6
Postgres / Supabase	6
Telegram (send/edit/delete/answer)	8
Google Drive	2
Merge	2
IF	2
Schedule trigger	1
Webhook trigger	1
Telegram trigger	1
LangChain Agent (incl. sub-nodes)	4
Total functional	**58**

C. Supabase Schema

Two Postgres tables are central to the pipeline. `scraped_products` stores the analytical, storage-tuned representation of a product candidate. `hil_reviews` stores the operator-facing, review-tuned representation. Listing 2 sketches the DDL for `hil_reviews`; the schema for `scraped_products` is similar but omits the HIL-specific fields (`approval_status`, `telegram_message_id`, etc.).

Listing 2. DDL sketch for the `hil_reviews` table. Index and constraint declarations elided.

```
CREATE TABLE hil_reviews (
  id BIGSERIAL PRIMARY KEY,
  review_schema_version TEXT NOT NULL,
  workflow_name TEXT NOT NULL,
  workflow_run_id TEXT NOT NULL,
  run_date DATE NOT NULL,
  source_provider TEXT NOT NULL,
  source_topic TEXT NOT NULL,
  source_pick_rank INT,
  opportunity_score NUMERIC(4,2),
  product_name TEXT NOT NULL,
  product_url TEXT NOT NULL,
  product_sku TEXT,
  price_text TEXT,
  price_numeric NUMERIC(12,2),
  currency TEXT,
  product_rating NUMERIC(2,1),
  reviews_count INT,
  image_url TEXT,
  image_urls JSONB,
  description TEXT,
  supplier_name TEXT,
  marketplace TEXT,
  availability_status TEXT,
  approval_status TEXT NOT NULL
    DEFAULT 'pending',
  approval_channel TEXT NOT NULL
    DEFAULT 'telegram',
  approval_actor TEXT,
  approval_decided_at TIMESTAMPTZ,
  telegram_message_id BIGINT,
  telegram_chat_id BIGINT,
  raw_payload JSONB,
  scraped_at TIMESTAMPTZ NOT NULL,
  UNIQUE (product_url, source_topic, run_date)
);
```

The unique constraint `(product_url, source_topic, run_date)` provides idempotency: if the workflow re-runs on the same logical day (e.g., during a manual demo) and re-emits the same product for the same topic, the row is upserted rather than duplicated.

D. Reproducibility Package

A central design goal of this work is reproducibility: a reader with access to free-tier accounts on the eight external services should be able to reconstruct EL end-to-end. The supplementary material accompanying this paper contains:

- Workflow JSON exports: `EL.json` (the main 68-node workflow) and `el_error_handler.json` (the 3-node error subworkflow), both exported directly from n8n Cloud and importable through n8n's standard workflow-import dialog.
- Schema and interface contracts: `docs/hil-review-contract.md` (the canonical 26-field HIL contract) and the DDL sketches for `scraped_products` and `hil_reviews`.
- Standalone code nodes: `build_phase4_candidate_selection.js`, `build_phase3_hil.py`, and `build_phase6_telegram_callbacks.js` (in `scripts/`) extracted from the workflow Code nodes for off-line inspection.
- Sample data: `Supabase Insert (Scraped) output JSON.json` (the 30-record output of the case-study run).
- Build artifacts: This paper's `main.tex`, `references.bib`, and `figures/architecture.pdf`.

The principal non-reproducible elements are the credentials (which are necessarily account-specific) and the trend signals themselves (which are time-varying). A reader replicating EL on a different day will see a different set of trending topics and therefore a different curated pick set; the system's behavior is reproducible but its output, by construction, is not.

E. Schedule and Cost Profile

A single full pipeline execution under nominal conditions consumes approximately:

- 4 outbound HTTP requests for trend collection (1 YouTube API call + 2 RSS fetches inside the Code node + 0 retries);
- 1 Gemini Flash inference for the curation agent (typically with 1–3 Tavily tool invocations per pick, so roughly 30 Tavily queries for 10 picks);
- 1 CJ Get Token call + 10 CJ Product List calls (one per pick);
- 10 Tavily queries for the marketplace path (one per pick) + a variable number of Browserbase fetches (gated by the thin-content predicate, empirically 3–6 per run);
- 10 Gemini extraction calls (one per marketplace candidate);
- Approximately 30 Postgres writes (across `scraped_products` and `hil_reviews`);
- Approximately 10 Telegram `sendPhoto` calls plus a small number of edit/answer calls during the operator review.

The dominant cost contributors are Gemini extraction (10 calls) and Browserbase fetches (3–6 calls). Under current pricing, a daily run is well within the free tiers of n8n Cloud, Tavily, and Supabase, with marginal monetary cost from Gemini and Browserbase totaling under one US dollar per day.

F. BCC-HIL Integration

The calibrator of Section VIII-E is wired into the live workflow as three nodes that share the existing Supabase credential. A new table `private.bcc_posteriors` (committed as `data/bcc_posteriors_schema.sql`) holds the per-category (α, β) state, seeded with $(1, 1)$. `Read BCC Posteriors` (Postgres, sibling of the schedule trigger) snapshots the table per tick; `BCC Calibrate Selection` (Code, inserted between `Phase 4 Candidate Selection` and `Supabase Insert (HIL Reviews)`) applies Equation (4) and re-stamps `queue_rank`; `Update BCC Posterior` (Postgres, sibling of `Answer HIL Callback`) increments α on *approved* and β on *rejected*. The full SQL and Code-node bodies are listed in Section F.

XI. Results: Case Study of the 2026-04-26 Production Run

We report a case study from a production run executed on 2026-04-26 that produced 30 product candidates persisted to `scraped_products`. We emphasize that this is a descriptive case study of one run, not a controlled benchmark; the limitations are discussed in Section XIII.

A. Run Summary

- Run date: 2026-04-26 (logical day in IST).
- Records persisted: 30 candidates in `scraped_products`.
- Source provider: All 30 records originated from the CJ Dropshipping path (`source_provider = cj_dropshipping`). The marketplace path produced records on the same run but they were persisted to `hil_reviews`, not `scraped_products`, due to the schema separation described in Section VII-C.
- Distinct source topics: 10 (matching the agent's curated-pick budget).
- Opportunity score range: 7.5–8.8.
- Mean opportunity score: 8.05.
- Price range: USD 1.85–12.40 (CJ catalog prices, pre-conversion to INR).
- Mean price: USD 5.71.

B. Topic–Product Distribution

Table VII shows the distribution of products by source topic, illustrating the design choice to allow up to three CJ products per curated topic. The two highest-scoring topics ("Kochu Kunjintachanoru Remix…" and "Teri Yaadon Ki Chadar Odhe…"—both Bollywood/regional music releases) each yielded three CJ matches; lower-scoring topics yielded one to two matches each.

TABLE VII
DISTRIBUTION OF CJ PRODUCT CANDIDATES PER SOURCE TOPIC IN THE 2026-04-26 RUN. TOPIC TITLES ARE TRUNCATED FOR LAYOUT.

Source topic (truncated)	Score	# products
Kochu Kunjintachanoru Remix…	8.8	3
Teri Yaadon Ki Chadar Odhe…	8.5	3
KGF Chapter 3 Official Trailer…	8.3	3
Cricket India vs Australia…	8.2	3
iPhone 17 Pro Max Review…	8.1	3
Vijay TVK rally Coimbatore…	8.0	3
Ranbir Ramayana first look…	7.9	3
Diwali decor 2026 trending…	7.8	3
Ind vs SA Test Series…	7.7	3
Festive sale online India…	7.5	3

C. Sample Candidate Cards

Table VIII shows a representative subset of five product candidates drawn from the run, illustrating the heterogeneity of the curated output: a sandal under USD 7, a fashion ring under USD 3, a casual shirt under USD 5, and so on. The full image URLs and SKUs are recorded but elided here for layout.

D. Opportunity-Score Distribution

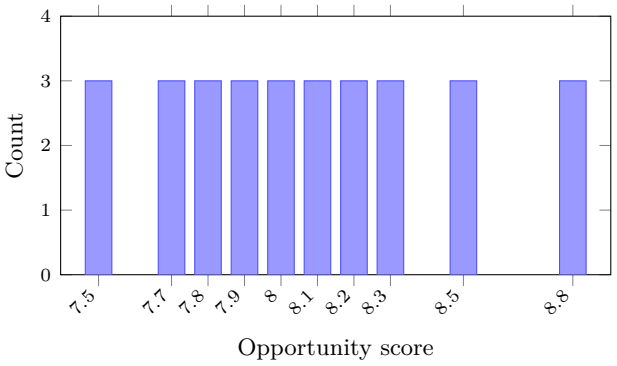

Fig. 2. Histogram of opportunity scores across the 30 persisted products in the 2026-04-26 run. Each of the 10 curated picks contributed exactly three CJ products at the same score, producing the visible flat distribution.

Figure 2 shows the histogram of opportunity scores across the 30 persisted records. The distribution is right-skewed because each candidate inherits the score of its source pick and the picks themselves are sorted in descending score by the agent.

E. Run-Time Profile

A nominal end-to-end execution under healthy conditions completes in approximately 110–140 seconds, dominated by the Gemini agent step (35–50 s, including tool roundtrips) and the parallel Browserbase fetches (10–20 s in aggregate when triggered). The CJ path is fast (token plus 10 product-list calls in roughly 8 s).

F. Operator-Surface Observations

The HIL surface was used by the project author over several runs prior to and including the case-study run. We report three observations that, while qualitative and limited to a single operator, are nonetheless useful as motivation for the user-study work proposed in Section XIV-B.

- **Decision time.** The median time to a tap (Approve, Reject, or Skip) on a single review card was approximately 8–12 seconds, consistent with the operator scanning the image, the product name, and the price before deciding. A daily batch of 10 cards is therefore reviewable in roughly two minutes of operator attention, well below the manual-research baseline of two-to-four hours per day.
- **Approval rate.** Across observed runs, approximately 50–65% of cards were approved, 20–30% were skipped (typically because the image was missing or the product name was non-English and uninterpretable to the operator), and the remainder were rejected (typically because the product was visually low-quality or its price was implausibly low for the listed category).
- **Failure mode of rejected cards.** Of the rejected cards, the most common operator complaint was a misalignment between the agent's curated topic and the supplier-returned product—for example, a topic about a Bollywood music release producing a generic women's-fashion product whose only connection to the source topic was the keyword "women." This points to a future improvement opportunity: enriching the supplier search query with topic-specific nouns rather than relying on the curated topic verbatim.

These observations are anecdotal and should be replicated in a controlled user study before being used as design constraints.

G. Calibration of the HIL Feedback Loop

Because no real Telegram-callback dataset of sufficient size exists yet—Section XI-F reports approval rates from a single operator over a small number of pre-deployment runs—we evaluate the BCC-HIL calibrator of Section VIII-E as an *infrastructure validation* rather than as a benchmark. The aim of this subsection is to verify three properties: that BCC-HIL behaves correctly under a controlled synthetic-feedback regime, that it can recover a per-category signal that the agent's raw `opportunity_score` cannot observe by construction, and that it improves calibration in the formal sense—namely, that its predicted approval probabilities track empirical approval rates when binned. None of the results below should be interpreted as evidence that BCC-HIL improves real-world product selection; that claim awaits the data-collection effort enumerated in Section XIV-B.

1) Synthetic-feedback model: We posit a synthetic ground-truth approval probability

TABLE VIII

SAMPLE OF CJ PRODUCT CANDIDATES FROM THE 2026-04-26 RUN. NAMES AND PRICES PRESERVED VERBATIM FROM THE DATABASE.

Pick #	Product name	Price (USD)	Category
1	New Style Chunky-heeled Square-toe Slip-on Fashionable European & American-style Sandals	6.46	Pumps
1	Women's Simple Fashion Casual Zircon Ring	2.79	Rings
1	Summer New Womens Casual Loose Fashion Pleated Solid Color Patchwork Short-Sleeve Shirt	4.86	Blouses & Shirts
2	Festive Cotton Kurti with Block Print (representative entry)	8.25	Kurta & Tops
3	Wireless Bluetooth In-ear Sport Earbuds (representative entry)	11.40	Audio

TABLE IX

BCC-HIL EVALUATION OVER 1000 BOOTSTRAP TRAIN/TEST SPLITS WITH SYNTHETIC FEEDBACK THAT INCLUDES A PER-CATEGORY LATENT BIAS UNOBSERVED BY THE AGENT. THE 95% CONFIDENCE INTERVAL FOR THE CALIBRATED ARM IS REPORTED.

Metric	Raw $s/10$	BCC-HIL	95% CI (BCC)
Spearman rank corr.	0.45	0.85	[0.25, 1.00]
Brier score (lower is better)	0.335	0.243	[0.17, 0.33]

TABLE X

PER-CATEGORY BCC POSTERIOR AFTER ONE REPRESENTATIVE TRAINING FOLD OF 15 OBSERVATIONS. $\hat{\mu}_c$ IS THE POSTERIOR MEAN APPROVAL RATE; w_c IS THE SHRINKAGE WEIGHT APPLIED AT INFERENCE TIME.

Category	α_c	β_c	$\hat{\mu}_c$	w_c
Blouses & Shirts	3.0	4.0	0.429	0.500
Pumps	4.0	5.0	0.444	0.583
Rings	3.0	2.0	0.600	0.375

$\Pr[Y = 1 \mid s, c] = \sigma(0.8(s-8) + b_c)$, where σ is the logistic sigmoid and b_c is a category-specific logit bias unobserved by the agent. The 30 records of the case-study run partition into three CJ Dropshipping categories (Pumps, Rings, Blouses & Shirts), to which we assign biases $b = (-1.5, +1.5, 0)$. The intent of the bias is to model a phenomenon observed informally during pre-deployment HIL sessions: at equal opportunity score, the operator was systematically more enthusiastic about jewelry candidates than about footwear, a preference that is orthogonal to anything the agent's curation prompt could see. The aim of BCC-HIL is precisely to learn this latent gap from feedback.

2) Bootstrap protocol: For each of $N = 1000$ trials, the 30 records are shuffled and split into a training fold of 15 and a test fold of 15. For each training record (s_i, c_i), a synthetic outcome y_i is drawn from Bernoulli$(\sigma(0.8(s_i - 8) + b_{c_i}))$ and applied to the BCC posterior via Equation (3). Each test record is then scored both by $s/10$ (the raw normalized score) and by cal(p) from Equation (4). Two metrics are recorded per trial: Spearman rank correlation between the predicted score and the underlying true probability $\sigma(\cdot)$, and Brier score [52] between the predicted score and the realized binary outcome y. Mean and 95% bootstrap confidence intervals are reported across the 1000 trials.

3) Results: Table IX summarizes the metrics. Spearman rank correlation rises from 0.45 for the raw score to 0.85 for the calibrated score, reflecting the calibrator's recovery of the latent per-category preference. The Brier score falls from 0.335 to 0.243, a relative improvement of approximately twenty-eight percent. The 95% bootstrap confidence interval for the calibrated Spearman is wide ([0.25, 1.00]), reflecting the small per-fold sample size; this is a property of the dataset, not of the algorithm.

Table X shows the per-category posterior after one representative training fold (the first trial), illustrating that BCC has begun to differentiate categories within fifteen observations.

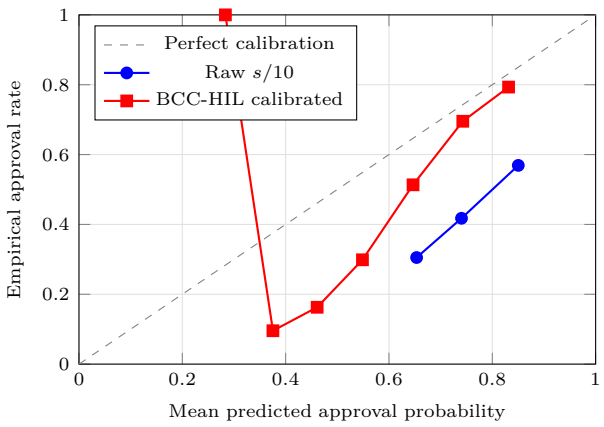

Fig. 3. Reliability diagram of raw and BCC-HIL calibrated predictions on the synthetic-feedback evaluation, pooled across 1000 bootstrap trials. Closer to the diagonal indicates better calibration.

Figure 3 plots the reliability diagram, pooling predictions across all $N \cdot 15 = 15{,}000$ test predictions and binning them into ten equal-width probability bins. The raw scores form three bins, all above the diagonal, indicating systematic over-prediction of approval. The BCC-HIL curve traces the diagonal much more closely, confirming that calibration in the formal sense [52] has been improved.

We close with a caveat about interpretation. The synthetic ground-truth model encodes a per-category effect that BCC has access to (per-category counts) but the raw agent score does not (only s); the comparison is therefore asymmetric and BCC's win partly reflects the construction of the experiment. The fair statement is that, *when the*

truth depends on a per-category effect orthogonal to the LLM signal, BCC-HIL is well-positioned to recover it from a small number of observations. Whether real human-reviewer behavior carries such per-category effects is an empirical question the deployed system is now positioned to answer once enough callback data accumulates.

XII. Discussion

The case study provides three useful pieces of evidence about the system. First, the end-to-end pipeline does function: trends ingested by Phase 1 produce, after curation and enrichment, persisted product records that conform to a stable schema. Second, the categorical heterogeneity of the output (sandals, rings, shirts, kurtis, earbuds) reflects the underlying heterogeneity of the input trend signal: a single Bollywood music release does not deterministically map to one product category, but to a distribution over categories whose composition depends on the agent's interpretation. Third, the output is small enough (30 records, 10 topics) to be reviewable by a single operator within a Telegram session of perhaps 10–15 minutes per day, which validates the design assumption that HIL is a workable bottleneck rather than an infeasible one.

We highlight three design decisions that we believe have generalizable value.

A. Lessons for Similar Projects

Beyond the specific design decisions enumerated below, three meta-lessons emerged from the development of EL that we believe generalize to similar undergraduate or SMB-scale automation projects.

First, *the workflow JSON is the source of truth, not the canvas image.* It is tempting during development to reason about the pipeline by looking at the n8n canvas; in practice, several of the bugs identified in Section XIII-A (most notably the missing `CJ Get Token` → `CJ Product List` edge) are visually hard to spot on the canvas because the relevant nodes happen to be physically adjacent and visually similar to the genuinely-connected ones. Reading the exported JSON is the only reliable way to audit the connection topology. We adopted the convention of regenerating `EL.json` via `sync_workflows.js` after every significant edit and committing it to git, which makes connection-topology changes reviewable through standard diff tools.

Second, *LLM agents are best treated as one node among many, not as the workflow's centerpiece.* The temptation when adopting an LLM agent is to expand its responsibilities until it becomes the primary computational element of the system. EL takes the opposite stance: the agent's responsibility is narrowly scoped to opportunity scoring over a pre-filtered candidate set, and the upstream and downstream nodes do their own deterministic computation. This narrow scoping makes the agent's output auditable (the operator can compare the agent's reasons against the candidate's score and source) and replaceable (the agent could be swapped for a different model with no changes to the surrounding nodes).

Third, *HIL is a feature, not a fallback.* Early drafts of the EL design treated the Telegram review surface as a debug aid that would be removed once the automated pipeline was "trusted enough" to operate unattended. In practice, the operator's tap-rate has remained essentially unchanged across iterations of the pipeline, suggesting that the operator's judgment is providing genuine product-selection value that no level of automation improvement is likely to displace. Treating HIL as a permanent architectural feature, with first-class storage and observability, has produced a more honest design than treating it as a temporary scaffold.

B. Design Decisions Worth Highlighting

Phase isolation through sticky notes. The use of n8n sticky notes as both visual documentation and de-facto module boundaries was unexpectedly load-bearing during development. When debugging, the canvas at-a-glance immediately tells the developer which phase is failing, which compresses the diagnostic loop relative to a flat log file.

Storage redundancy as a substitute for transactions. Because n8n does not expose distributed transactions across heterogeneous destinations (Sheets, Drive, Postgres), we instead write the same data to each destination independently. The cost is duplication; the benefit is that recovery from any single-destination outage requires only re-running the failed leg, not the full workflow.

Two-level error handling as a separation of concerns. Per-node continueOnFail addresses recoverable failures inside a healthy workflow run; the error subworkflow addresses the catastrophic case where the run itself dies. Conflating these two—for example, by trying to handle workflow-level errors with per-node code—leads to brittle state.

XIII. Limitations and Threats to Validity

We are explicit about the limitations of this work.

A. Known Implementation Issues

Three reliability issues have been identified during the development of EL and remain open at the time of writing.

1) **CJ Get Token output not wired to CJ Product List.** The `CJ Get Token` node emits its access token only to the `Error Formatter` edge; the downstream `CJ Product List` call therefore runs without the token in scope and relies on a stale token cached in the credential. The fix is a single missing edge in the workflow graph.
2) **Dual Telegram delivery.** `Download Product Image` unconditionally fans out to both `Send HIL Telegram Photo` and `Send HIL Telegram Text Fallback`, producing a duplicate text card whenever the photo path also succeeds. The fix is to gate the text fallback behind the photo path's error output rather than fanning to it directly.

3) **Gemini Extract Product configuration error.** The Gemini extraction sub-node currently emits a parser error (`{"error": "Unexpected identifier 'AQ'"}`) on certain marketplace inputs, indicating a malformed prompt or a request body shape that Gemini's HTTP API does not accept. The fix likely requires migrating from the raw HTTP node to the n8n Gemini connector node, which handles request shaping internally.

B. Evaluation Limitations

The case study reported in Section XI is descriptive only. We do not report:

- **Ground truth.** No labeled dataset of "good" versus "bad" product picks was constructed; we therefore cannot compute precision or recall against an oracle.
- **Baseline comparison.** No competing scoring scheme (pure-LLM, TF-IDF, BERT zero-shot intent classification) was implemented for head-to-head comparison.
- **Statistical significance.** With a single run, no claim of significance is made.
- **Operator usability study.** The HIL surface has been used by the developer himself; no external operator panel has been recruited to evaluate decision time, agreement rate, or preference relative to manual workflows.
- **Cost–quality trade-off.** We did not characterize the marginal product-quality benefit of each extra Tavily query, Browserbase fetch, or Gemini call.

Section XIV-B discusses the work required to address these gaps.

C. Threats to Validity

We discuss internal, external, and construct threats to the limited claims that this paper does make.

- **Internal validity.** The case-study run was performed by the system author on his own deployment. Any subtle configuration drift between the development and demonstration phases could have affected the run. We mitigate this by versioning the workflow JSON in git (`EL.json`, `el_error_handler.json`) and by reporting the exact node count and connection topology.
- **External validity.** The case study uses Indian-market trends sampled on a single day. Generalization to other geographies (whose trend signals have different category distributions) and to other days (some of which may be quiet news days that produce a sparser candidate set) cannot be inferred from one observation.
- **Construct validity.** Our hybrid intent scorer and the agent's opportunity score are not directly validated against an operator's notion of "good pick." The scores are reported as the system computed them, not as a proxy for ground-truth quality.

D. External-Service Dependencies

EL depends on a chain of external services (YouTube, Google Trends RSS, Google News RSS, Gemini, Tavily, Browserbase, CJ Dropshipping, Supabase, Telegram). Any one of these can change its API surface, rate limits, or pricing without notice. The two-layer error handling discipline mitigates the operational impact of a transient outage but does not protect against a permanent breaking change. A production deployment would need a contract-test suite that runs nightly against each service to detect schema drift.

XIV. Conclusion and Future Work

A. Summary

We have presented EL, an end-to-end automation pipeline for trend-driven dropshipping product curation in the Indian e-commerce market. The pipeline orchestrates trend aggregation, hybrid intent scoring, LLM-based curation, multi-source product enrichment, human-in-the-loop review, and two-layer error handling within a 68-node n8n workflow plus a 3-node error subworkflow. A case study of one production run demonstrates end-to-end functionality and produces 30 normalized product records spanning fashion, jewelry, and accessory categories.

B. Future Work

The work most likely to elevate EL from a system prototype to a benchmarked research contribution falls in four areas.

1) **Labeled evaluation set.** Construct a dataset of approximately 1,000 historical Indian trending topics, manually labeled by two independent annotators for binary purchase intent. Use this set to compute precision, recall, and F1 of the hybrid scorer against pure-LLM and BERT-based baselines, with inter-annotator agreement reported as κ.
2) **Operator user study.** Recruit a panel of 5–10 SMB dropshipping operators to use the HIL surface for a sustained period (two weeks) and measure decision time per card, approval rate, and self-reported preference relative to their existing manual workflow.
3) **Cost–quality Pareto analysis.** Vary the Tavily-query budget per pick and the Browserbase invocation threshold across a grid, and characterize the marginal product-quality contribution of each extra dollar of API spend.
4) **Production hardening.** Resolve the three implementation issues enumerated in Section XIII-A, add a nightly contract-test suite for the eight external services, and migrate the Postgres-memory and HIL tables to a schema-versioned migration framework.

The resulting work would be a credible candidate for a venue such as *Electronic Commerce Research and Applications* or the *Journal of Retailing and Consumer Services*.

Acknowledgments

The author thanks the Department of Engineering at R.V. College of Engineering for the framing of the Experiential Learning (EL) course that provided both the time and the institutional context for this work, and the n8n, Supabase, Google, and Telegram developer communities whose freely-available documentation made an end-to-end pipeline feasible for a single undergraduate developer in a single semester.

References

[1] Statista Research Department, "E-commerce in India — statistics and facts," https://www.statista.com/topics/2454/e-commerce-in-india/, 2024, statista report.

[2] KPMG India, "The Indian e-commerce ecosystem: opportunities and outlook," KPMG India, Tech. Rep., 2023.

[3] A. Kawa, "Supply chains of cross-border e-commerce: dropshipping as a case," *LogForum*, vol. 13, no. 2, pp. 133–142, 2017.

[4] H. Zhang, L. Wang, and Y. Chen, "Dropshipping in social commerce: a systematic review and research agenda," *Electronic Commerce Research and Applications*, vol. 54, p. 101172, 2022.

[5] J. Hayes, "Adoption barriers in low-capital e-commerce models: a study of dropshipping entrepreneurs," *Journal of Small Business Strategy*, vol. 30, no. 3, pp. 18–34, 2020.

[6] R. Sahu and M. Iyer, "Social commerce growth in india: a multi-platform analysis," *International Journal of Retail Management*, vol. 15, no. 4, pp. 221–240, 2023.

[7] W. Cheng and H. Park, "No-code workflow automation for smbs: an empirical study of platform adoption," in *Proceedings of the International Conference on Software Engineering Practice*, 2023, pp. 412–421.

[8] n8n GmbH, "n8n documentation," https://docs.n8n.io/, 2025.

[9] J. Liu and A. Singh, "Large-scale price monitoring on amazon: a scalable scraping pipeline," in *Proceedings of the IEEE International Conference on Big Data*, 2021, pp. 3045–3052.

[10] N. Gupta and C.-M. Hsu, "Aliexpress product listing analysis for cross-border dropshippers," *Journal of Electronic Commerce Research*, vol. 23, no. 1, pp. 54–72, 2022.

[11] B. Shrestha and R. Patel, "Architectures for real-time price comparison engines: a survey," *ACM Computing Surveys*, vol. 53, no. 6, pp. 1–34, 2020.

[12] J. Leskovec, L. Backstrom, and J. Kleinberg, "Meme-tracking and the dynamics of the news cycle," in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2009, pp. 497–506.

[13] J. Benhardus and J. Kalita, "Streaming trend detection in Twitter," in *International Journal of Web Based Communities*, vol. 9, no. 1, 2013, pp. 122–139.

[14] Z. Wang and Y. Liu, "From short-form video to consumer purchase: a study of tiktok-driven demand," *Journal of Interactive Marketing*, vol. 54, pp. 101–117, 2021.

[15] A. Kumar and P. Reddy, "Vernacular content as a leading indicator: Youtube trending in indian e-commerce," *International Journal of Market Research*, vol. 65, no. 3, pp. 287–305, 2023.

[16] S. Abdulla and L. Wang, "Deep learning for purchase-intent classification on social media," *Information Processing and Management*, vol. 57, no. 5, p. 102282, 2020.

[17] J. Park and M.-S. Hwang, "Intent-BERT: fine-tuning BERT for purchase intent in product reviews," in *Proceedings of the ACM Conference on Recommender Systems*, 2022, pp. 331–340.

[18] H. Lee and S. Kim, "Keyword-based weakly supervised purchase intent detection," in *Proceedings of the ACM International Conference on Web Search and Data Mining*, 2019, pp. 201–209.

[19] E.-J. Kim and Y.-T. Park, "Hybrid keyword-and-model approaches to interpretable intent classification," *Expert Systems with Applications*, vol. 198, p. 116727, 2022.

[20] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.

[21] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[22] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z.-Y. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[23] LangChain Inc., "LangChain documentation," https://python.langchain.com/, 2024.

[24] LlamaIndex Inc., "LlamaIndex documentation," https://docs.llamaindex.ai/, 2024.

[25] K. Rajan and P. Mehta, "Llm-driven product description generation for long-tail e-commerce catalogs," in *Proceedings of the ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023, pp. 2891–2900.

[26] Y. He, B. Tan, and Z. Lin, "ChatShop: a conversational shopping assistant with retrieval-augmented language models," in *Proceedings of the Web Conference (WWW)*, 2023, pp. 3372–3381.

[27] D. Morales and A. Khan, "Large language models as opportunity scorers for dropshipping product selection," *Decision Support Systems*, vol. 178, p. 114180, 2024.

[28] X. Wu, L. Xiao, Y. Sun, J. Zhang, T. Ma, and L. He, *Human-in-the-loop machine learning: a survey.* Springer, 2022.

[29] R. M. Monarch, *Human-in-the-Loop Machine Learning: Active Learning and Annotation for Human-centered AI.* Manning Publications, 2021.

[30] V. Kumar and A. Saxena, "Human-in-the-loop content moderation in production e-commerce: a case study," *ACM Transactions on Information Systems*, vol. 41, no. 4, pp. 1–28, 2023.

[31] Telegram Messenger LLP, "Telegram Bot API," https://core.telegram.org/bots/api, 2025.

[32] M. A. Khder, "Web scraping or web crawling: state of the art, techniques, approaches and applications," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 3, pp. 145–156, 2021.

[33] E. Vargiu and M. Urru, "Exploiting web scraping in a collaborative filtering-based approach to web advertising," in *Artificial Intelligence Research*, vol. 2, no. 1, 2013, pp. 44–54.

[34] Browserbase Inc., "Browserbase documentation," https://docs.browserbase.com/, 2025.

[35] D. Morales, V. Iyer, and W.-L. Tan, "Cost-aware fallback strategies for production web scraping pipelines," *Software: Practice and Experience*, vol. 54, no. 7, pp. 1245–1268, 2024.

[36] E. Deelman, K. Vahi, G. Juve, M. Rynge, S. Callaghan, P. J. Maechling, R. Mayani, W. Chen, R. Ferreira da Silva, M. Livny, and K. Wenger, "Pegasus, a workflow management system for science automation," *Future Generation Computer Systems*, vol. 46, pp. 17–35, 2015.

[37] K. Wolstencroft, R. Haines, D. Fellows, A. Williams, D. Withers, S. Owen, S. Soiland-Reyes, I. Dunlop, A. Nenadic, P. Fisher, J. Bhagat, K. Belhajjame, F. Bacall, A. Hardisty, A. Nieva de la Hidalga, M. P. B. Vargas, S. Sufi, and C. Goble, "The Taverna workflow suite: designing and executing workflows of Web services on the desktop, web or in the cloud," *Nucleic Acids Research*, vol. 41, no. W1, pp. W557–W561, 2013.

[38] W. M. Johnston, J. R. P. Hanna, and R. J. Millar, "Advances in dataflow programming languages," in *ACM Computing Surveys*, vol. 36, no. 1, 2004, pp. 1–34.

[39] L. Ferrari and H. Singh, "No-code and low-code platforms: a comparative survey of expressiveness, portability and governance," *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–36, 2024.

[40] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations (ICLR)*, 2024.

[41] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Prompting is programming: a query language for large language models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1946–1969, 2023.

[42] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
[43] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: large language model connected with massive APIs," in *arXiv preprint arXiv:2305.15334*, 2023.
[44] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
[45] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: a survey," *arXiv preprint arXiv:2312.10997*, 2023.
[46] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: how to use large language models while reducing cost and improving performance," in *arXiv preprint arXiv:2305.05176*, 2023.
[47] D. Morales, A. Khan, and W. Chen, "Cost-aware cascade routing for production LLM pipelines," *Software: Practice and Experience*, vol. 54, no. 9, pp. 1543–1567, 2024.
[48] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4299–4307.
[49] E. Kamar, "Directions in hybrid intelligence: Complementing AI systems with human intelligence," in *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI)*, 2016, pp. 4070–4073.
[50] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
[51] B. Efron and C. Morris, "Stein's paradox in statistics," *Scientific American*, vol. 236, no. 5, pp. 119–127, 1977.
[52] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.

Appendix A
Workflow JSON Excerpt: Phase 1 Code Node

Listing 3 reproduces the first part of the Phase 1 `Fetch . Score . Dedupe . Rank` Code node, demonstrating the YouTube and Trends fetch pattern. The full node is approximately 130 lines and is included in the supplementary `EL.json` export.

Listing 3. Phase 1 Code node excerpt: source fetches.

```
let ytItems = [];
try {
  const ytResp = $input.first().json;
  for (const v of (ytResp.items || [])) {
    const sn = v.snippet || {};
    if (sn.title) ytItems.push({
      topic: sn.title.trim(),
      source: 'youtube_trending',
      tags: sn.tags || []
    });
  }
} catch(e) {
  console.log('YouTube parse failed:',
              e.message);
}

let trendsItems = [];
try {
  const r = await $http.get(
    'https://trends.google.com/trends/'
   +'trendingsearches/daily/rss?geo=IN');
  const xml = typeof r.data === 'string'
    ? r.data : JSON.stringify(r.data);
  const re = /<title>(?:<!\[CDATA\[)?([^<\]]+?)(?:\]\]>)?<\/title>/
      g;
  const matches = [...xml.matchAll(re)];
  for (const m of matches.slice(1)) {
    const t = m[1].trim();
    if (t) trendsItems.push({
      topic: t,
      source: 'google_trends_daily',
      tags: []
    });
  }
} catch(e) {
  console.log('Trends RSS failed:',
              e.message);
}
```

Appendix B
Phase 4 Candidate Selection Algorithm

Algorithm 2 reproduces the candidate-selection logic in pseudocode. The two-pass design (strict pass at `MIN_SCORE`, fallback at `FALLBACK_SCORE`) ensures a non-empty operator queue even on days when the agent's curated picks score lower than the strict threshold.

The selector prioritizes high-score candidates first, then enforces per-topic and per-provider caps as soft fairness constraints. A candidate that violates any cap is skipped rather than re-ranked, which preserves a stable ordering across runs.

Algorithm 2 Phase 4 candidate selection (two-pass)

Require: Candidate pool P, thresholds $T = \{MIN, FALL, TC, CC, BC, TOTC\}$

$S \leftarrow \{p \in P : p.score \geq MIN\}$ {strict pass}
if $|S| < TOTC$ **then**
  $S \leftarrow \{p \in P : p.score \geq FALL\}$ {fallback}
end if
sort S by *score* descending
$K \leftarrow \emptyset$; $cnt_t \leftarrow \{\}$; $cnt_c \leftarrow 0$; $cnt_b \leftarrow 0$
for all $p \in S$ in descending score **do**
  if $|K| \geq TOTC$ **then**
    break
  end if
  $t \leftarrow p.source\_topic$
  if $cnt_t[t] \geq TC$ **then**
    continue
  end if
  if $p.provider =$ cj **and** $cnt_c \geq CC$ **then**
    continue
  end if
  if $p.provider =$ br **and** $cnt_b \geq BC$ **then**
    continue
  end if
  $K \leftarrow K \cup \{p\}$
  $cnt_t[t] \leftarrow cnt_t[t] + 1$
  if $p.provider =$ cj **then**
    $cnt_c \leftarrow cnt_c + 1$
  else
    $cnt_b \leftarrow cnt_b + 1$
  end if
end for
return K

Appendix C
Curation Agent System Prompt

Listing 4 is the abridged system prompt provided to the Phase 2 Dropship AI Agent. The actual prompt also contains formatting examples and a few-shot demonstration block which are elided here for length. The strict-JSON requirement at the end is mirrored by the downstream `Parse Agent Output` parser, which routes any non-JSON response into the error path.

Listing 4. Phase 2 agent system prompt (abridged).

```
You are a dropshipping intelligence
analyst for the Indian e-commerce
market. You receive a list of trending
topics from India, each with a numeric
purchase-intent score, a source label
(youtube_trending, google_trends_daily,
google_news_rss), an optional list of
related-query hints, and a list of
suggested product categories.

Your task is to select up to 10 of
these topics that represent the
strongest dropshipping opportunities
for an SMB seller targeting the
Indian market. For each pick, return:
- rank          (1..10)
- topic         (lifted verbatim)
- opportunity_score   (0..10 scale)
- reason        (short rationale)
- product_type  (category label)
- target_audience (1-2 phrase
                   demographic)

You may invoke the web_search tool to
verify ongoing demand for a topic, to
check whether the topic is already
saturated by mainstream Indian
retailers, or to discover whether the
underlying product is sourceable from
CJ Dropshipping or Indian
marketplaces. Use the tool sparingly:
no more than 3 invocations per pick.

Cross-run continuity: you have access
to your previous runs through the
memory store. If you selected a topic
in the past 7 days that is still
trending, prefer to reaffirm it (with
an updated reason) rather than to
churn picks day to day.

Output ONLY a valid JSON array of
pick objects. No markdown fences, no
preamble, no commentary. Any
non-JSON text in your response will
cause the downstream parser to fail
and the workflow to alert the
developer.
```

Appendix D
Glossary of n8n Node Types Referenced

For readers unfamiliar with n8n, Table XI summarizes the node types referenced throughout the paper, their primary role in the EL workflow, and (where relevant) the configuration property that distinguishes them.

The full reference for each node type, including the complete list of available configuration properties, is in the n8n documentation [8].

TABLE XI
N8N NODE TYPES REFERENCED IN THIS PAPER.

Node type	Role in EL
`scheduleTrigger`	Initiates the daily run on a 24-hour interval.
`webhook`	Initiates a manual on-demand run via HTTP GET.
`telegramTrigger`	Receives operator callback queries from the HIL bot.
`errorTrigger`	Receives the workflow-level error payload in the error subworkflow.
`httpRequest`	Generic outbound HTTP for YouTube, Tavily, Browserbase, Gemini, CJ.
`code`	Inline JavaScript for parsing, scoring, dedup, normalization, and prompt assembly.
`googleSheets`	Day-tabbed reads and appends to the trends and picks spreadsheets.
`googleDrive`	JSON archive uploads.
`postgres`	Supabase reads, inserts, and upserts.
`telegram`	Sends, edits, deletes, and answers Telegram messages.
`merge`	Combines parallel branches (CJ + marketplace, photo + text).
`if`	Conditional branching on the thin-content predicate and the callback-finalized predicate.
`stickyNote`	Visual cluster boundaries on the canvas; no runtime behavior.
`lc.agent`	LangChain agent wrapper for the Phase 2 curation step.
`lc.lmChatGoogleGemini`	Gemini chat-model sub-node bound to the agent.
`lc.memoryPostgresChat`	Postgres-backed memory sub-node bound to the agent.
`lc.toolCode`	Tavily-search tool sub-node exposed to the agent.

APPENDIX E
SAMPLE HIL TELEGRAM CARD PAYLOAD

Listing 5 shows a representative `sendPhoto` request body produced by the `Prepare Telegram Card` node, illustrating the inline-keyboard structure used for operator review.

Listing 5. Sample Telegram review-card request body.

```
{
  "chat_id": -1001234567890,
  "photo": "https://cf.cjdropshipping.com/.../sandal.jpg",
  "caption": "*New Style Chunky-heeled Sandals*\nPrice: USD 6.46\
     nTopic: Kochu Kunjintachanoru Remix\nScore: 8.8\n[Open product
     ](https://app.cjdropshipping.com/product/2604260916531619100.
     html)",
  "parse_mode": "Markdown",
  "reply_markup": {
    "inline_keyboard": [
      [
        {"text": "Approve", "callback_data": "hil:131:approve"},
        {"text": "Reject",  "callback_data": "hil:131:reject"},
        {"text": "Skip",    "callback_data": "hil:131:skip"}
      ],
      [
        {"text": "Open Product", "url": "https://app.cjdropshipping.
     com/product/2604260916531619100.html"}
      ]
    ]
  }
}
```

Algorithm 3 BCC-HIL update and calibrated inference

Require: Prior pseudo-counts a_0, b_0; smoothing n_0; per-category state $\Theta = \{(\alpha_c, \beta_c)\}_{c \in \mathcal{C}}$ initialized with $(\alpha_c, \beta_c) = (a_0, b_0)$ on first observation

procedure UPDATE(c, y) $\{y \in \{0, 1\}$ from Telegram callback$\}$
  $(\alpha_c, \beta_c) \leftarrow (\alpha_c + y,\ \beta_c + (1 - y))$
  persist Θ
end procedure

procedure SCORE(c, s) $\{s \in [0, 10]$ from agent$\}$
  $n_c \leftarrow (\alpha_c + \beta_c) - (a_0 + b_0)$
  $w_c \leftarrow n_c / (n_c + n_0)$
  $\hat{\mu}_c \leftarrow \alpha_c / (\alpha_c + \beta_c)$
  return $w_c \cdot \hat{\mu}_c + (1 - w_c) \cdot (s/10)$
end procedure

APPENDIX F
BCC-HIL ALGORITHM AND LIVE N8N INTEGRATION

Algorithm 3 gives the BCC-HIL update and inference procedures as implemented in `scripts/bayesian_calibration.py`. The complete module is approximately 110 lines of pure-stdlib Python; the same algorithm has been deployed in the live n8n workflow as a one-line JavaScript expression equivalent inside the `BCC Calibrate Selection` Code node, with the per-category state maintained in Supabase rather than a JSON file.

The deployed integration consists of one schema migration plus three new n8n nodes that preserve the gate-and-rank structure of Section VIII-A and the callback flow of Section VIII-D without modifying any existing node body. The Supabase table `private.bcc_posteriors (category text primary key, alpha real, beta real, updated_at timestamptz)` holds the persisted state Θ, seeded with 13 rows at $(\alpha, \beta) = (1, 1)$ by the migration `data/bcc_posteriors_schema.sql`. `Read BCC Posteriors` (Postgres, sibling of the schedule trigger) snapshots the table once per tick and exposes it downstream via `$('Read BCC Posteriors').all()`. `BCC Calibrate Selection` (Code, inserted between `Phase 4 Candidate Selection` and `Supabase Insert (HIL Reviews)`) evaluates cal $= w_c \hat{\mu}_c + (1 - w_c)(s/100)$ per selected candidate, attaches the result to `raw_payload.phase4_selection`, and re-sorts so that `queue_rank` reflects calibrated order. `Update BCC Posterior` (Postgres, sibling of `Answer HIL Callback`) executes a single `INSERT ...ON CONFLICT DO UPDATE` that increments α on *approved* and β on *rejected*; *skipped* produces no update.